

Informe del Código: Positional vs Symbolic Attention Heads

Análisis detallado de la implementación completa

Positional versus Symbolic Attention Heads: Learning Dynamics, RoPE Geometry, and Length Generalization — Urrutia et al. (2026)

Reconstrucción basada en el paper original

14 de julio de 2026

Índice general

0.1.	Estructura Completa del Código	2
0.2.	Generación de Datasets	2
0.2.1.	vocab.py	2
0.2.2.	number_task.py	3
0.2.3.	letter_task.py	4
0.3.	Modelo: GPT-J con Single Head	5
0.3.1.	config.py	5
0.3.2.	gptj.py	5
0.3.3.	layers.py	6
0.4.	Entrenamiento	8
0.4.1.	train.py	8
0.4.2.	eval.py	9
0.5.	Métricas: Positional y Symbolic Scores	10
0.5.1.	permutation_utils.py	10
0.5.2.	positional_symbolic_scores.py	10
0.5.3.	entropia.py	12
0.6.	Análisis	13
0.6.1.	head_purity.py	13
0.6.2.	frequency_analysis.py	14
0.6.3.	discrepancy.py	15
0.7.	Construcciones Teóricas	16
0.7.1.	index_construction.py	16
0.7.2.	retrieval_construction.py	17
0.8.	Visualizaciones	19
0.8.1.	learning_dynamics.py	19
0.8.2.	attention_patterns.py	19
0.8.3.	geometric_analysis.py	20
0.8.4.	generalization_plots.py	21
0.9.	Evaluación en LLMs	21
0.9.1.	prompt_templates.py	21
0.9.2.	run_llm_tests.py	22
0.10.	Flujo de Ejecución Completo	24
0.11.	Dependencias	24
0.12.	Notas sobre la Reconstrucción	25

0.1. Estructura Completa del Código

Este documento explica en detalle todos los componentes del código utilizado en el paper “Positional versus Symbolic Attention Heads: Learning Dynamics, RoPE Geometry, and Length Generalization”.

Basado en la descripción del paper, el código se organiza en los siguientes módulos:

```
number-and-letter-neurips26-C754/
+-- LICENSE                                # MIT License
+-- README.md                             # Descripción del repositorio
+-- src/
    +-- cmds/                             # Puntos de entrada (scripts ejecutables)
        | +-- multihop_exp.py              # Experimento Number Task (entrenamiento)
        | +-- symbolic_exp.py              # Experimento Letter Task (entrenamiento)
        | +-- get_attention_weights.py      # Extracción de atención con permutación
        | +-- compute_metrics.py           # Cálculo positional/symbolic scores
        | +-- compute_metrics_multiple_queries.py # Scores para múltiples posiciones que
        | +-- compute_symbolic_metrics.py   # Scores para Letter Task
        | +-- schemas.py                   # Esquemas Pydantic de configuración
    +-- data/                             # Generación de datasets y tokenizadores
        | +-- dataset.py                   # Number Task: SequenceMultiHopDataset
        | +-- symbolic_dataset.py           # Letter Task: SequenceSymbolicDataset
        | +-- generate_dataset_sym.py       # Helper para generar Letter dataset
        | +-- utils.py                     # Validación de secuencias
    +-- lib/                              # Librerías compartidas
        | +-- metrics.py                   # Positional/Symbolic scores
        | +-- s3.py                        # Subida/descarga a S3
    +-- models/                           # Entrenamiento personalizado
        +-- train.py                       # Trainer + Callbacks + Métricas
```

0.2. Generación de Datasets

0.2.1. vocab.py

Propósito: Define los vocabularios para ambas tareas y las funciones de tokenización.

Componentes:

- ALPHABET: Conjunto de letras
 - Number Task: 26 letras + 94 bigramas (aa a dp) = 120 símbolos
 - Letter Task: 8 letras {a, b, c, d, e, f, g, h}
- INTEGERS: Conjunto de enteros
 - Number Task: {1, 2, ..., 16}
 - Letter Task: {1, 2, ..., 8}
- NUM_VOCAB: 136 tokens únicos (Number)

- LET_VOCAB: 128 tokens únicos (Letter)
- `tokenize(sequence)`: Convierte secuencia de tokens a índices
- `detokenize(indices)`: Convierte índices a tokens

Detalle de implementación:

```
1 # Number Task vocabulary
2 # Tokens de letras: a-z (26) + aa, ab, ..., dp (94 bigramas)
3 # Tokens de números: 1, 2, ..., 16
4 # Total: 120 + 16 = 136 tokens
5
6 # Letter Task vocabulary
7 # Target window: pares letra-número (8 x 8 = 64 tokens atomicos)
8 # Pre-target window: pares letra-letra (8 x 8 = 64 tokens atomicos)
9 # Total: 64 + 64 = 128 tokens
```

0.2.2. number_task.py

Propósito: Genera el dataset para la Number Task.

Proceso de generación:

1. Parámetros:

- `n1 = 8` (target window length)
- `n2 = 8` (context/pre-target length)
- `max_hops = 4`
- `N = 480,000` (total sequences)

2. Algoritmo de generación:

```
1 def generate_number_sequence(hops, target_letter, target_pos):
2     """
3     Genera una secuencia valida para Number Task.
4
5     Args:
6         hops: int (1-4), numero de hops
7         target_letter: str, letra a recuperar (respuesta)
8         target_pos: int, posicion de la letra en target window
9
10    Returns:
11        sequence: list[str], secuencia de tokens
12    """
13    # Target window: letras + numeros de hop
14    target_window = [...]
15
16    # Pre-target window: numeros de hop
17    pre_target = [random_int() for _ in range(n2)]
18
19    # Asegurar que la secuencia de hops esta bien definida
20    # Siguiendo la Definicion B.1 del paper:
21    # 1.  $j_{k1} = j_{n2}$  (ultimo entero)
22    # 2.  $n_1 - j_{kHL} + 1 \in \{1, \dots, n_1\}$ 
23    # 3. Para cada  $i$ :  $k_i - k_{(i+1)} = j_{ki}$ 
```

```

24
25     return target_window + pre_target

```

3. **Validación:** Cada secuencia debe tener un camino de hops único y bien definido.
4. **Balanceo:** Igual proporción de 1, 2, 3, 4 hops. Para cada hop class, balanceo de valor de solución y posición.
5. **Split:** 90 % train (432,000), 0.5 % validation (2,400), 9.5 % test (45,600)

0.2.3. letter_task.py

Propósito: Genera el dataset para la Letter Task.

Proceso de generación:

1. Parámetros:

- ALPH = {a, b, c, d, e, f, g, h} (8 letras)
- INT = {1, 2, ..., 8} (8 enteros)
- n1 = 8 (target window)
- n2 = 8 (pre-target window)
- N = 480,000

2. Algoritmo:

```

1  def generate_letter_sequence(hops):
2      """
3      Genera una secuencia valida para Letter Task.
4
5      Args:
6          hops: int, numero de hops
7
8      Returns:
9          sequence: list[tuple], pares (letra, letra) o (letra, numero)
10     """
11     # Target window: secuencia de pares (letra, numero)
12     target_window = [(letter, num) for _ in range(n1)]
13
14     # Pre-target window: pares (letra, letra)
15     # Cada par (X,Y) indica: busca el par con primera letra = Y
16     pre_target = []
17     for h in range(hops):
18         pair = (random_letter(), random_letter())
19         pre_target.append(pair)
20
21     # Validar que el camino es unico:
22     # y_ki = x_k(i+1) (segunda letra del par i = primera letra del par
23     # i+1)
24     # y_kHL = s_i* (ultima segunda letra apunta a target)
25     return target_window + pre_target

```

3. **Misma estructura de datos:** 480k secuencias, 17 tokens, split 90/0.5/9.5

0.3. Modelo: GPT-J con Single Head

0.3.1. config.py

Propósito: Define la configuración del modelo.

```
1 model_config = {
2     "layers": 12,           # Numero de capas
3     "d_model": 128,        # Dimension oculta
4     "heads": 1,            # 1 attention head por capa (single-head)
5     "d_head": 128,         # Dimension por head = d_model (single head
6                             )
7     "d_ff": 512,           # Dimension feed-forward (4x d_model)
8     "vocab_size": 136,     # Number Task: 136; Letter Task: 128
9     "max_seq_len": 17,     # Longitud de secuencia fija
10    "dropout_rate": 0.1,    # Dropout
11    "activation": "gelu",   # GELU activation
12    "use_rope": True,       # Rotary Positional Encoding
13    "rope_base": 10000,     # Base frequency for RoPE
14 }
```

0.3.2. gptj.py

Propósito: Implementación del modelo GPT-J.

Arquitectura:

```
1 class GPTJModel(nn.Module):
2     """
3     Decoder-only Transformer basado en GPT-J.
4
5     Capas:
6     1. Embedding: token -> vector d_model
7     2. 12 x TransformerBlock
8         - LayerNorm
9         - SingleHeadAttention (con RoPE)
10        - Residual connection
11        - LayerNorm
12        - MLP (GELU activacion)
13        - Residual connection
14    3. Final LayerNorm
15    4. LM Head (proyeccion a vocab_size)
16    """
17
18    def __init__(self, config):
19        self.embedding = nn.Embedding(config.vocab_size, config.d_model)
20
21        self.blocks = nn.ModuleList([
22            TransformerBlock(config) for _ in range(config.layers)
23        ])
24
25        self.ln_f = nn.LayerNorm(config.d_model)
26        self.lm_head = nn.Linear(config.d_model, config.vocab_size,
27                                   bias=False)
28
29        # Atar pesos de embedding y lm_head
30        self.lm_head.weight = self.embedding.weight
```

```

29     def forward(self, input_ids):
30         """
31         Forward pass con causal masking.
32
33         Args:
34             input_ids: (batch, seq_len)
35
36         Returns:
37             logits: (batch, seq_len, vocab_size)
38             attention_maps: list de matrices de atencion por capa
39         """
40         x = self.embedding(input_ids)
41         attention_maps = []
42
43         for block in self.blocks:
44             x, attn = block(x) # attn: (batch, heads, seq_len, seq_len)
45             attention_maps.append(attn)
46
47         x = self.ln_f(x)
48         logits = self.lm_head(x)
49
50         return logits, attention_maps

```

0.3.3. layers.py

Propósito: Implementación de las capas individuales.

SingleHeadAttention con RoPE:

```

1  class SingleHeadAttention(nn.Module):
2      """
3      Atencion single-head con RoPE (Rotary Positional Encoding).
4
5      Arquitectura estandar:
6      - Q, K, V proyecciones lineales
7      - RoPE aplicado a Q y K
8      - Scaled dot-product attention con causal mask
9      - Output projection
10     """
11
12     def __init__(self, config):
13         self.d_model = config.d_model
14         self.d_head = config.d_head # = d_model (single head)
15
16         # Proyecciones
17         self.q_proj = nn.Linear(d_model, d_head, bias=False)
18         self.k_proj = nn.Linear(d_model, d_head, bias=False)
19         self.v_proj = nn.Linear(d_model, d_head, bias=False)
20         self.out_proj = nn.Linear(d_head, d_model, bias=False)
21
22         # RoPE frecuencias
23         # d_head/2 frecuencias: theta_k = base^(-2k/d_head)
24         self.rope_frequencies = self._compute_rope_frequencies()
25
26     def _compute_rope_frequencies(self):
27         """Computa frecuencias RoPE."""

```

```

28     freqs = []
29     for k in range(self.d_head // 2):
30         theta = 10000.0 ** (-2.0 * k / self.d_head)
31         freqs.append(theta)
32     return torch.tensor(freqs)
33
34     def _apply_rope(self, x, positions):
35         """Aplica RoPE a las activaciones."""
36         # x: (batch, seq_len, d_head)
37         # positions: (seq_len,) indices de posicion
38
39         # Para cada par (x_2k, x_(2k+1)):
40         # RoPE(x, pos)_2k = x_2k * cos(pos*theta_k) - x_(2k+1) * sin(
41             pos*theta_k)
42         # RoPE(x, pos)_(2k+1) = x_2k * sin(pos*theta_k) + x_(2k+1) *
43             cos(pos*theta_k)
44
45         cos, sin = self._compute_rope_cache(positions)
46         x_rotated = ...
47         return x_rotated
48
49     def forward(self, x, mask=None):
50         """
51         Args:
52             x: (batch, seq_len, d_model)
53
54         Returns:
55             output: (batch, seq_len, d_model)
56             attention_weights: (batch, seq_len, seq_len)
57         """
58         batch, seq_len, _ = x.shape
59
60         # Proyecciones
61         q = self.q_proj(x) # (batch, seq_len, d_head)
62         k = self.k_proj(x)
63         v = self.v_proj(x)
64
65         # RoPE
66         positions = torch.arange(seq_len, device=x.device)
67         q = self._apply_rope(q, positions)
68         k = self._apply_rope(k, positions)
69
70         # Scaled dot-product attention
71         #  $L(q_i, k_j) = (R_j k_j)^T (R_i q_i) = k_j^T R^{(i-j)} q_i$ 
72         scores = torch.matmul(q, k.transpose(-2, -1)) / math.sqrt(self.
73             d_head)
74
75         # Causal mask
76         if mask is not None:
77             scores = scores.masked_fill(mask == 0, float('-inf'))
78
79         # Softmax
80         attention_weights = F.softmax(scores, dim=-1)
81
82         # Weighted sum
83         output = torch.matmul(attention_weights, v)
84
85         # Output projection

```



```

83         output = self.out_proj(output)
84
85         return output, attention_weights

```

TransformerBlock:

```

1  class TransformerBlock(nn.Module):
2      def __init__(self, config):
3          self.ln1 = nn.LayerNorm(config.d_model)
4          self.attention = SingleHeadAttention(config)
5          self.ln2 = nn.LayerNorm(config.d_model)
6          self.mlp = nn.Sequential(
7              nn.Linear(config.d_model, config.d_ff),
8              nn.GELU(),
9              nn.Linear(config.d_ff, config.d_model),
10             nn.Dropout(config.dropout_rate),
11         )
12
13     def forward(self, x):
14         # Pre-norm + attention + residual
15         attn_out, attn_weights = self.attention(self.ln1(x))
16         x = x + attn_out
17
18         # Pre-norm + MLP + residual
19         x = x + self.mlp(self.ln2(x))
20
21     return x, attn_weights

```

0.4. Entrenamiento

0.4.1. train.py

Propósito: Loop de entrenamiento con last-token prediction.

```

1  def train(model, dataloader, optimizer, num_epochs):
2      """
3      Entrenamiento con last-token prediction.
4
5      La perdida solo se calcula en el ultimo token de cada secuencia.
6      Los tokens anteriores tienen ignore_index = -100 (no contribuyen al
7      gradiente).
8      """
9      for epoch in range(num_epochs):
10         for batch in dataloader:
11             input_ids, labels = batch
12             # input_ids: (batch, seq_len)
13             # labels: (batch, seq_len) con -100 en todos excepto ultimo
14             # token
15
16             logits, _ = model(input_ids)
17
18             # Solo ultimo token contribuye a la perdida
19             loss = F.cross_entropy(
20                 logits[:, -1, :], # Solo ultimo paso de tiempo
21                 labels[:, -1],    # Solo etiqueta del ultimo token
22                 ignore_index=-100

```

```
21         )
22
23         loss.backward()
24         optimizer.step()
25         optimizer.zero_grad()
```

Justificación: El objetivo es que el modelo aprenda a predecir el token correcto al final de la secuencia (respuesta de la tarea multi-hop), ignorando los otros tokens. Esto fuerza al modelo a desarrollar mecanismos internos para resolver la tarea.

0.4.2. eval.py

Propósito: Evaluación en test set.

```
1 def evaluate(model, test_loader):
2     """
3     Evalua la precision del modelo.
4     - Accuracy global
5     - Accuracy por hop (1, 2, 3, 4)
6     """
7     model.eval()
8     correct = 0
9     total = 0
10    hop_correct = {1: 0, 2: 0, 3: 0, 4: 0}
11    hop_total = {1: 0, 2: 0, 3: 0, 4: 0}
12
13    with torch.no_grad():
14        for batch in test_loader:
15            input_ids, labels, hop_counts = batch
16            logits, _ = model(input_ids)
17
18            # Prediccion en el ultimo token
19            predictions = logits[:, -1, :].argmax(dim=-1)
20
21            # Comparar con etiquetas
22            for i, (pred, label, hops) in enumerate(
23                zip(predictions, labels[:, -1], hop_counts)
24            ):
25                if pred == label:
26                    correct += 1
27                    hop_correct[hops.item()] += 1
28                total += 1
29                hop_total[hops.item()] += 1
30
31    accuracy = correct / total
32    hop_accuracies = {
33        h: hop_correct[h] / hop_total[h]
34        for h in range(1, 5) if hop_total[h] > 0
35    }
36
37    return accuracy, hop_accuracies
```

0.5. Métricas: Positional y Symbolic Scores

0.5.1. permutation_utils.py

Propósito: Funciones para permutar secuencias y calcular pesos.

```
1 def get_swap_permutation(i, j, n):
2     """
3     Crea una permutacion que intercambia las posiciones i y j.
4
5     Args:
6         i, j: indices a intercambiar (0-indexed)
7         n: longitud total de la secuencia
8
9     Returns:
10        perm: lista de indices permutados
11    """
12    perm = list(range(n))
13    perm[i], perm[j] = perm[j], perm[i]
14    return perm
15
16 def calculate_swap_weight(attention_weights, i, j, temperature=1.0):
17     """
18     Calcula el peso alpha(pi) para el swap entre i y j.
19
20     El peso depende de cuanta masa de atencion se mueve en el swap.
21
22     Args:
23         attention_weights: distribucion de atencion en el query final
24         i, j: posiciones a intercambiar
25         temperature: temperatura del softmax (tau)
26
27     Returns:
28         weight: peso de esta permutacion
29    """
30    di = attention_weights[i]
31    dj = attention_weights[j]
32    return F.softmax(torch.tensor([di - dj]) / temperature).item()
```

0.5.2. positional_symbolic_scores.py

Propósito: Implementación de las métricas positional y symbolic scores.

```
1 def compute_positional_score(head_attention, hidden_states):
2     """
3     Calcula el positional score para un head en un input dado.
4
5     El positional score mide cuan invariante es la atencion
6     ante permutaciones de los key vectors.
7
8     Args:
9         head_attention: distribucion de atencion del ultimo token
10        hidden_states: representaciones ocultas en cada posicion
11
12    Returns:
13        pos_score: float entre 0 y 1
14    """
```

```

15     n = len(hidden_states)
16     pos_score = 0.0
17
18     for i in range(n - 1):
19         for j in range(i + 1, n):
20             # Atencion original (query en n, keys en i, j)
21             v_ij = (head_attention[i], head_attention[j])
22
23             # Permutar estados ocultos en posiciones i, j
24             hidden_permuted = hidden_states.clone()
25             hidden_permuted[i], hidden_permuted[j] = \
26                 hidden_permuted[j], hidden_permuted[i]
27
28             # Re-evaluar atencion con estados permutados
29             # (en la practica se estima desde las activaciones)
30             attention_permuted = reestimate_attention(
31                 hidden_permuted, head_attention
32             )
33             v_ij_perm = (attention_permuted[i], attention_permuted[j])
34
35             # Peso de la permutacion
36             alpha = compute_swap_weight(head_attention, i, j)
37
38             # Similitud coseno entre atencion original y permutada
39             cos_sim = cosine_similarity(v_ij, v_ij_perm)
40             pos_score += alpha * cos_sim
41
42     return pos_score / total_weight
43
44 def compute_symbolic_score(head_attention, hidden_states):
45     """
46     Calcula el symbolic score para un head en un input dado.
47
48     El symbolic score mide cuan equivariante es la atencion
49     ante permutaciones de los key vectors.
50     """
51     n = len(hidden_states)
52     sym_score = 0.0
53
54     for i in range(n - 1):
55         for j in range(i + 1, n):
56             v_ij = (head_attention[i], head_attention[j])
57
58             # Bajo equivariancia: v_ij(pi(x)) = v_ji(x)
59             # La atencion en i, j despues de permutar = atencion
60             # original en j, i
61             v_ji = (head_attention[j], head_attention[i])
62
63             alpha = compute_swap_weight(head_attention, i, j)
64             cos_sim = cosine_similarity(v_ij, v_ji)
65             sym_score += alpha * cos_sim
66
67     return sym_score / total_weight
68
69 def compute_scores_for_model(model, dataset):
70     """
71     Calcula positional y symbolic scores para todos los heads
72     en todo el dataset.

```

```

72
73     Para cada capa:
74     - Extraer la atencion del ultimo token
75     - Calcular positional score promedio sobre todas las secuencias
76     - Calcular symbolic score promedio sobre todas las secuencias
77
78     Returns:
79         pos_scores: list[float] por capa
80         sym_scores: list[float] por capa
81         entropy: list[float] entropia de atencion por capa
82     """
83     pos_scores = []
84     sym_scores = []
85     entropy = []
86
87     for layer_idx in range(model.config.layers):
88         layer_pos = []
89         layer_sym = []
90         layer_entropy = []
91
92         for sequence in dataset:
93             # Forward pass hasta esta capa
94             _, attentions = model.forward_until(sequence, layer_idx)
95             head_attn = attentions[layer_idx][0] # single head
96
97             # Ultimo token como query
98             last_query_attn = head_attn[-1, :] # atencion desde ultimo
99             token
100
101             # Scores
102             pos = compute_positional_score(last_query_attn,
103                                           model.get_hidden_states(
104                                               sequence, layer_idx))
105
106             sym = compute_symbolic_score(last_query_attn,
107                                         model.get_hidden_states(
108                                             sequence, layer_idx))
109
110             # Entropia normalizada
111             ent = compute_entropy(last_query_attn)
112
113             layer_pos.append(pos)
114             layer_sym.append(sym)
115             layer_entropy.append(ent)
116
117         pos_scores.append(np.mean(layer_pos))
118         sym_scores.append(np.mean(layer_sym))
119         entropy.append(np.mean(layer_entropy))
120
121     return pos_scores, sym_scores, entropy

```

0.5.3. entropia.py

Propósito: Cálculo de entropía de la distribución de atención.

```

1 def compute_entropy(attention_weights):
2     """
3     Calcula la entropia normalizada de la distribucion de atencion.

```

```

4
5 Entropia = -Sigma p_i * log(p_i) / log(n)
6 Normalizada: 0 = determinista, 1 = uniforme
7
8 Args:
9     attention_weights: distribucion de atencion (ultimo token)
10
11 Returns:
12     normalized_entropy: float entre 0 y 1
13     """
14     # Entropia de Shannon
15     entropy = -torch.sum(attention_weights * torch.log(
16         attention_weights + 1e-10))
17
18     # Normalizar por log(n)
19     n = attention_weights.shape[-1]
20     normalized = entropy / math.log(n)
21
22     return normalized.item()

```

0.6. Análisis

0.6.1. head_purity.py

Propósito: Análisis de pureza de heads (cuándo un head se vuelve claramente posicional o simbólico).

```

1 def classify_head_purity(pos_score, sym_score, gamma=0.1):
2     """
3     Clasifica un head como puro o mixto.
4
5     Args:
6         pos_score: positional score (0-1)
7         sym_score: symbolic score (0-1)
8         gamma: umbral de tolerancia
9
10    Returns:
11        'positional': si pos_score > 1-gamma y sym_score < gamma
12        'symbolic': si sym_score > 1-gamma y pos_score < gamma
13        'mixed': en otro caso
14    """
15    if pos_score > (1 - gamma) and sym_score < gamma:
16        return 'positional'
17    elif sym_score > (1 - gamma) and pos_score < gamma:
18        return 'symbolic'
19    else:
20        return 'mixed'
21
22 def compute_purity_dynamics(model, dataset, checkpoints, gamma=0.1):
23     """
24     Computa la dinamica de pureza a traves del entrenamiento.
25
26     Para cada checkpoint, cuenta:
27     - Numero de heads posicionales puros
28     - Numero de heads simbolicos puros
29

```

```

30     Returns:
31         timeline: {step: {'pos': count, 'sym': count}}
32     """
33     timeline = {}
34
35     for step, checkpoint in enumerate(checkpoints):
36         model.load_state_dict(checkpoint)
37         pos_scores, sym_scores, _ = compute_scores_for_model(model,
38                                     dataset)
39
40         pos_count = 0
41         sym_count = 0
42         for pos, sym in zip(pos_scores, sym_scores):
43             classification = classify_head_purity(pos, sym, gamma)
44             if classification == 'positional':
45                 pos_count += 1
46             elif classification == 'symbolic':
47                 sym_count += 1
48
49         timeline[step] = {'pos': pos_count, 'sym': sym_count}
50
51     return timeline

```

0.6.2. frequency_analysis.py

Propósito: Análisis de uso de frecuencias RoPE por capa.

```

1  def analyze_frequency_use(model):
2      """
3      Analiza que frecuencias RoPE usa cada capa.
4
5      Para cada head, identifica la frecuencia dominante
6      (la que mas contribuye a la distribucion de atencion).
7
8      Returns:
9          frequency_map: {layer_idx: dominant_freq_id}
10     """
11     frequency_map = {}
12
13     for layer_idx, block in enumerate(model.blocks):
14         attn = block.attention
15
16         # Obtener los pesos de Q y K
17         W_q = attn.q_proj.weight
18         W_k = attn.k_proj.weight
19
20         # Para cada frecuencia RoPE (d_head/2 frecuencias)
21         # Calcular que frecuencia tiene mayor magnitud en Q*K^T
22         freq_contributions = []
23         for freq_idx in range(model.config.d_head // 2):
24             # Extraer los componentes 2D para esta frecuencia
25             q_freq = W_q[:, 2*freq_idx:2*freq_idx+2]
26             k_freq = W_k[:, 2*freq_idx:2*freq_idx+2]
27
28             # Norma de Frobenius como proxy de contribucion
29             contribution = torch.norm(q_freq @ k_freq.T)
30             freq_contributions.append(contribution.item())

```

```

31
32     dominant_freq = np.argmax(freq_contributions)
33     frequency_map[layer_idx] = dominant_freq
34
35     return frequency_map

```

0.6.3. discrepancy.py

Propósito: Cálculo de la discrepancia teórica para los mecanismos Index y Retrieval.

```

1  def index_discrepancy_upper_bound(n, theta):
2      """
3      Cota superior de discrepancia para Index (Theorem 3.2).
4
5      Delta_Index(n) <= min{1 - cos(theta), pi^2 / (2n^2)}
6
7      Args:
8          n: longitud de la secuencia
9          theta: angulo RoPE usado
10
11      Returns:
12          bound: float, cota superior
13      """
14      bound1 = 1 - math.cos(theta)
15      bound2 = math.pi**2 / (2 * n**2)
16      return min(bound1, bound2)
17
18  def retrieval_discrepancy_upper_bound(n, theta, omega, alphabet_size):
19      """
20      Cota superior de discrepancia para Retrieval (Theorem 3.2).
21
22      Delta_Retrieval(n) <= 1 - cos(omega - n*theta)
23      donde omega = 2*pi/|ALPH|
24
25      Args:
26          n: longitud de la secuencia
27          theta: angulo RoPE
28          omega: angulo de codificacion de simbolos
29          alphabet_size: |ALPH|
30
31      Returns:
32          bound: float, cota superior
33      """
34      omega = 2 * math.pi / alphabet_size
35      bound = 1 - math.cos(omega - n * theta)
36      return bound
37
38  def compute_theoretical_discrepancies(max_len=300):
39      """
40      Computa las discrepancias teoricas para el rango de longitudes.
41
42      Returns:
43          index_bounds: list[float]
44          retrieval_bounds: list[float]
45      """
46      # Parametros del modelo entrenado (Tabla 1 del paper)

```



```

47 # Layer 2 en Number Task: theta = theta_2 = 0.8 (freq_id=2, freq
    =0.8660)
48 # Layer 2 en Letter Task: theta = theta_42 = 0.0027 (freq_id=42,
    freq=0.0027)
49 theta_index = 0.8 # Angulo para Index
50 theta_retrieval = 0.0027 # Angulo para Retrieval
51 alphabet_size = 8 # |ALPH| = 8 para Letter Task
52
53 index_bounds = []
54 retrieval_bounds = []
55
56 for n in range(1, max_len + 1):
57     idx_bound = index_discrepancy_upper_bound(n, theta_index)
58     ret_bound = retrieval_discrepancy_upper_bound(
59         n, theta_retrieval, None, alphabet_size
60     )
61     index_bounds.append(idx_bound)
62     retrieval_bounds.append(ret_bound)
63
64 return index_bounds, retrieval_bounds

```

0.7. Construcciones Teóricas

0.7.1. index_construction.py

Propósito: Implementación de la construcción teórica de Index (Theorem 3.1).

```

1 def construct_index_transformer(v, theta, alphabet, integers):
2     """
3     Construye un Transformer T_Index que implementa Index.
4
5     Arquitectura:
6     - Embedding: one-hot + bias column
7     - W_Q: query vectors = v para letras,  $\rho(\theta)^{-\sigma}v$  para
        numeros
8     - W_K: key vectors = v para todos
9     - W_V: diagonal,  $\alpha > 1$  para coordenadas de letras, 0 para otras
10    - RoPE: un solo angulo theta
11
12    Args:
13        v: vector 2D no nulo
14        theta: angulo RoPE
15        alphabet: conjunto de letras
16        integers: conjunto de enteros
17
18    Returns:
19        T_Index: transformer construido
20    """
21    d = len(alphabet) + len(integers) + 1 # +1 para bias
22
23    # W_Q in  $\mathbb{R}^{2 \times d}$ 
24    W_Q = torch.zeros(2, d)
25    for sigma in alphabet:
26        # Para letras:  $W_Q \cdot E(\sigma) = v$ 
27        W_Q[:, sigma_idx] = v
28    for sigma in integers:

```

```

29         # Para numeros: W_Q*E(sigma) = rho(theta)^(-sigma)*v
30         rotation = rotation_matrix(-sigma * theta)
31         W_Q[:, sigma_idx] = rotation @ v
32
33         # W_K in R^{2*d}
34         W_K = torch.zeros(2, d)
35         for sigma in alphabet + integers:
36             W_K[:, sigma_idx] = v
37
38         # W_V in R^{d*d}, diagonal
39         W_V = torch.zeros(d, d)
40         for sigma in alphabet:
41             # alpha > 1 para coordenadas de letras
42             W_V[sigma_idx, sigma_idx] = 2.0
43
44         return T_Index(W_Q, W_K, W_V, theta)
45
46 def index_attention_logit(query_token, key_token, query_pos, key_pos,
47     theta):
48     """
49     Logit de atencion para Index.
50
51     Si el query es letra:
52         L = cos((n - j)*theta)
53     Si el query es numero:
54         L = cos((n - j - sigma_n)*theta)
55
56     Donde sigma_n es el valor del numero en la posicion query.
57     """
58     if is_letter(query_token):
59         return math.cos((query_pos - key_pos) * theta)
60     else: # es numero
61         offset = int(query_token)
62         return math.cos((query_pos - key_pos - offset) * theta)
63
64 def index_max_attended_position(query_token, query_pos, theta):
65     """
66     Determina la posicion de atencion maxima para Index.
67
68     Si query es letra: atiende a si mismo (posicion query_pos)
69     Si query es numero con valor sigma: atiende a posicion (query_pos -
70         sigma)
71     """
72     if is_letter(query_token):
73         return query_pos
74     else:
75         offset = int(query_token)
76         return query_pos - offset

```

0.7.2. retrieval_construction.py

Propósito: Implementación de la construcción teórica de Retrieval.

```

1 def construct_retrieval_transformer(v, omega, alphabet, integers):
2     """
3     Construye un Transformer T_Retrieval que implementa Retrieval.
4

```

```

5     Arquitectura:
6     - Embedding: one-hot sobre pares (letra_izq, letra_der/o numero)
7     - W_Q: query vectors codifican la letra derecha (o izquierda si es
      target)
8     - W_K: key vectors codifican la letra izquierda
9     - W_V: identity (copia el valor)
10    - RoPE: angulo theta, con  $0 < \theta < \omega/2n_0$ 
11
12    Args:
13        v: vector 2D
14        omega: angulo =  $2\pi/|\text{ALPH}|$ 
15        alphabet: conjunto de letras
16        integers: conjunto de enteros
17
18    Returns:
19        T_Retrieval: transformer construido
20    """
21    d = 2 * len(alphabet) + len(integers)
22
23    # Para un par (L, R):
24    # W_Q * E(L,R) =  $\rho(\omega)^{-\phi(R)} * v$  si L,R in ALPH (pre-target)
25    # W_Q * E(L,R) =  $\rho(\omega)^{-\phi(L)} * v$  si L in ALPH, R in INT (
      target)
26    # W_K * E(L,R) =  $\rho(\omega)^{\phi(L)} * v$  siempre
27
28    W_Q = torch.zeros(2, d)
29    W_K = torch.zeros(2, d)
30    # ... (construccion similar a Index pero con codificacion simbolica
      )
31
32    W_V = torch.eye(d) # Identity
33
34    return T_Retrieval(W_Q, W_K, W_V, omega)
35
36 def retrieval_attention_logit(query, key, query_pos, key_pos, theta,
      omega):
37     """
38     Logit de atencion para Retrieval.
39
40     L =  $\cos((\phi(L_{\text{key}}) - \phi(R_{\text{query}})) * \omega + (n - j) * \theta)$ 
41
42     donde phi mapea letras a indices numericos.
43     """
44     L_key = get_left_letter(key)
45     R_query = get_right_letter(query)
46
47     phi_L = letter_to_index(L_key)
48     phi_R = letter_to_index(R_query)
49
50     angle = (phi_L - phi_R) * omega + (query_pos - key_pos) * theta
51     return math.cos(angle)

```

0.8. Visualizaciones

0.8.1. learning_dynamics.py

Propósito: Generar las visualizaciones de la Figura 2.

```

1 def plot_learning_dynamics(accuracy_history, pos_scores_history,
2                             sym_scores_history, entropy_history,
3                             purity_history):
4     """
5     Genera la Figura 2 del paper.
6
7     Panel A: Task accuracy por hop y global
8     Panel B: Positional y symbolic scores por capa
9     Panel C: Head purity counts
10    """
11    fig, axes = plt.subplots(3, 2, figsize=(12, 15))
12
13    # A: Accuracy
14    for hop in [1, 2, 3, 4]:
15        axes[0, 0].plot(accuracy_history['hop_{}'.format(hop)],
16                        label='Hop {}'.format(hop))
17    axes[0, 0].plot(accuracy_history['global'], 'm--', linewidth=2,
18                    label='Global')
19
20    # B: Scores por capa
21    for layer, pos, sym, ent in zip(
22        range(len(pos_scores_history[0])),
23        pos_scores_history, sym_scores_history, entropy_history
24    ):
25        axes[1, 0].plot(pos, 'r-', label='Pos Layer {}'.format(layer))
26        axes[1, 0].plot(sym, 'g-', label='Sym Layer {}'.format(layer))
27        axes[1, 0].plot(ent, 'm--', label='Entropy')
28
29    # C: Purity
30    axes[2, 0].plot(purity_history['pos'], 'r-', label='Pure positional')
31    axes[2, 0].plot(purity_history['sym'], 'g-', label='Pure symbolic')
32
33    plt.tight_layout()
34    plt.savefig('fig2_learning_dynamics.png', dpi=300)

```

0.8.2. attention_patterns.py

Propósito: Visualizar patrones de atención (Figura 3).

```

1 def plot_attention_patterns(model, sequence, task_type):
2     """
3     Visualiza los patrones de atencion para todas las capas.
4
5     Muestra:
6     - Para cada capa: quien atiende a quien
7     - Las posiciones de los tokens
8     - Identificacion de Index, Retrieval, Reflexive
9     """
10    _, attentions = model(sequence)
11

```

```

12 fig, axes = plt.subplots(4, 3, figsize=(15, 20))
13
14 for layer_idx, attn in enumerate(attention):
15     ax = axes[layer_idx // 3, layer_idx % 3]
16
17     # Matriz de atencion (causal)
18     attn_matrix = attn[0].detach().numpy() # single head
19
20     im = ax.imshow(attn_matrix, cmap='viridis', aspect='auto')
21     ax.set_title('Layer {}'.format(layer_idx))
22
23     # Anotar nombres de tokens
24     ax.set_xticks(range(len(sequence)))
25     ax.set_yticks(range(len(sequence)))
26     ax.set_xticklabels(sequence, rotation=90, fontsize=6)
27     ax.set_yticklabels(sequence, fontsize=6)
28
29 plt.tight_layout()
30 plt.savefig('fig3_attention_patterns.png', dpi=300)

```

0.8.3. geometric_analysis.py

Propósito: Visualización geométrica de mecanismos RoPE (Figura 4).

```

1 def plot_geometric_analysis(model, sequence):
2     """
3     Visualiza la geometria de los vectores query y key en el
4     plano rotacional de RoPE.
5
6     Panel A: Construcccion teorica de Index
7     Panel B: Vectores del modelo entrenado
8     Panel C: Frecuencia dominante RoPE
9     """
10    fig, axes = plt.subplots(1, 3, figsize=(15, 5))
11
12    # A: Construcccion teorica
13    # Key vectors todos alineados
14    # Query vectors rotados segun posicion
15
16    # B: Modelo entrenado
17    # Proyectar Q y K al plano de la frecuencia dominante
18    dominant_freq, q_proj, k_proj = extract_rope_projection(model,
19                                                            sequence)
20
21    axes[0].scatter(q_proj[:, 0], q_proj[:, 1], c='blue', label='
22    Queries')
23    axes[0].scatter(k_proj[:, 0], k_proj[:, 1], c='red', label='Keys')
24
25    # C: Max logit por frecuencia
26    freq_contributions = compute_frequency_contributions(model,
27                                                            sequence)
28    axes[2].bar(range(len(freq_contributions)), freq_contributions)
29    axes[2].axvline(dominant_freq, color='r', linestyle='--')
30
31    plt.tight_layout()
32    plt.savefig('fig4_geometric_analysis.png', dpi=300)

```

0.8.4. generalization_plots.py

Propósito: Visualización de resultados de generalización (Figura 5).

```

1 def plot_generalization_results(index_discrepancy,
2     retrieval_discrepancy,
3     model_accuracy_num, model_accuracy_let,
4     llm_results):
5     """
6     Genera la Figura 5 del paper.
7
8     Panel A: Cotas teoricas de discrepancia
9     Panel B: Precision del modelo entrenado
10    Panel C: Precision de GPT 5.4 y 5.5
11    Panel D: Precision de Claude Sonnet 3.7
12    """
13    fig, axes = plt.subplots(2, 2, figsize=(12, 10))
14
15    # A: Discrepancia teorica
16    lengths = range(1, 101)
17    axes[0, 0].plot(lengths, index_discrepancy, 'r-', label='Index')
18    axes[0, 0].plot(lengths, retrieval_discrepancy, 'g-', label='
19    Retrieval')
20
21    # B: Modelo entrenado
22    axes[0, 1].plot(model_accuracy_num, 'r-', label='Number task')
23    axes[0, 1].plot(model_accuracy_let, 'g-', label='Letter task')
24
25    # C: LLMs
26    for model_name, results in llm_results.items():
27        axes[1, 0].errorbar(results['lengths'], results['num_means'],
28            yerr=results['num_stds'], fmt='o-',
29            label='{} Number'.format(model_name))
30        axes[1, 1].errorbar(results['lengths'], results['let_means'],
31            yerr=results['let_stds'], fmt='s-',
32            label='{} Letter'.format(model_name))
33
34    plt.tight_layout()
35    plt.savefig('fig5_generalization.png', dpi=300)

```

0.9. Evaluación en LLMs

0.9.1. prompt_templates.py

Propósito: Templates de prompts para evaluar LLMs en los tasks 1-hop.

```

1 NUMBER_TASK_PROMPT = """You will be given a sequence of space-separated
2     tokens.
3 Your task is to find the target token by following these rules:
4 1. Identify the very last token in the sequence, which will always be a
5     number 'n'.
6 2. Move 'n' positions to the left from that final token.
7 3. The token you land on is the target token.
8 Output just the target token and absolutely nothing else. Do not
9     include any explanations or formatting.
10
11 Example 1:

```

```

9  Input: "a z b y c x d w 5"
10 Output: y
11
12 Example 2:
13 Input: "v t k g n k o h m p 4"
14 Output: o
15
16 Example 3:
17 Input: "w y b c v t r i p p 10"
18 Output: w
19
20 Task:
21 Input: "{sequence}"
22 Output: ""
23
24 LETTER_TASK_PROMPT = ""You will be given a sequence of space-separated
    tokens.
25 Your task is to find the target token by following these rules:
26 1. Start with the very last token in the sequence.
27 2. Identify the second character of that token.
28 3. Scan backwards (to the left) to find the token that starts with that
    exact character; there is only one token that fulfills this
    condition.
29 4. This is your target token.
30 Output just the target token and absolutely nothing else. Do not
    include any explanations or formatting.
31
32 Example:
33 Input: "a4 b3 c2 d1 fc"
34 Output: c2
35
36 Input: "w9 t7 l5 g1 n4 l6 u9 k7 m5 p8 gk"
37 Output: k7
38
39 Input: "q5 r7 x4 t7 f4 k7 q4 u2 u3 r5 ex"
40 Output: x4
41
42 Task:
43 Input: "{sequence}"
44 Output: ""

```

0.9.2. run_llm_tests.py

Propósito: Ejecutar evaluación en LLMs (GPT 5.4, GPT 5.5, Claude Sonnet 3.7).

```

1  def evaluate_llm_on_task(llm_client, task_type, sequence,
    prompt_template):
2      """
3      Evalua un LLM en una instancia de la tarea.
4
5      Args:
6          llm_client: API client del LLM
7          task_type: 'number' o 'letter'
8          sequence: secuencia de tokens
9          prompt_template: template de prompt
10
11      Returns:

```

```
12         answer: str, respuesta del modelo
13         correct: bool, si la respuesta es correcta
14     """
15     prompt = prompt_template.format(sequence=sequence)
16
17     response = llm_client.complete(
18         prompt=prompt,
19         max_tokens=1, # Forzar respuesta de un token
20         temperature=0.0,
21         stop=None
22     )
23
24     answer = response.strip()
25     correct_answer = compute_correct_answer(sequence, task_type)
26
27     return answer, answer == correct_answer
28
29 def run_generalization_experiment(llm_clients, task_types,
30     sequence_lengths,
31                                     n_runs=5):
32     """
33     Ejecuta el experimento completo de generalizacion.
34
35     Para cada modelo, tarea y longitud:
36     - Generar n_runs datasets diferentes
37     - Evaluar precision
38     - Reportar media y desviacion estandar
39     """
40     results = {}
41
42     for model_name, client in llm_clients.items():
43         model_results = {'number': {}, 'letter': {}}
44
45         for task_type in ['number', 'letter']:
46             for length in sequence_lengths:
47                 accuracies = []
48
49                 for run in range(n_runs):
50                     # Generar dataset para esta longitud
51                     dataset = generate_dataset(task_type, length, seed=
52                         run)
53
54                     correct = 0
55                     for sequence in dataset:
56                         answer, is_correct = evaluate_llm_on_task(
57                             client, task_type, sequence,
58                             get_prompt_template(task_type)
59                         )
60                         if is_correct:
61                             correct += 1
62
63                     accuracy = correct / len(dataset)
64                     accuracies.append(accuracy)
65
66                 mean_acc = np.mean(accuracies)
67                 std_acc = np.std(accuracies)
68                 model_results[task_type][length] = (mean_acc, std_acc)
```



```
68         results[model_name] = model_results
69
70     return results
```

0.10. Flujo de Ejecución Completo

El pipeline completo del paper se ejecuta en el siguiente orden:

```
1  # 1. Generar datasets
2  python dataset/number_task.py --output data/number_task/
3  python dataset/letter_task.py --output data/letter_task/
4
5  # 2. Entrenar modelos
6  python training/train.py --config configs/number_task.yaml --output
   models/number_model/
7  python training/train.py --config configs/letter_task.yaml --output
   models/letter_model/
8
9  # 3. Calcular metricas (positional/symbolic scores)
10 python metrics/positional_symbolic_scores.py --model models/
    number_model/ --dataset data/number_task/
11 python metrics/positional_symbolic_scores.py --model models/
    letter_model/ --dataset data/letter_task/
12
13 # 4. Analisis de pureza de heads
14 python analysis/head_purity.py --scores results/scores_number.pkl
15 python analysis/head_purity.py --scores results/scores_letter.pkl
16
17 # 5. Analisis de frecuencias RoPE
18 python analysis/frequency_analysis.py --model models/number_model/
19 python analysis/frequency_analysis.py --model models/letter_model/
20
21 # 6. Pruebas de generalizacion
22 python analysis/generalization.py --model models/number_model/ --
    dataset data/number_task/
23 python analysis/generalization.py --model models/letter_model/ --
    dataset data/letter_task/
24
25 # 7. Calculo de discrepancia teorica
26 python analysis/discrepancy.py --theta 0.8 --omega 0.785 --alphabet-
    size 8
27
28 # 8. Evaluacion en LLMs
29 python llm_evaluation/run_llm_tests.py --models gpt-5.4,gpt-5.5,claude-
    sonnet-3.7
30
31 # 9. Generar visualizaciones
32 python visualization/learning_dynamics.py
33 python visualization/attention_patterns.py
34 python visualization/geometric_analysis.py
35 python visualization/generalization_plots.py
```

0.11. Dependencias

```
1 # Core
2 torch>=2.0.0
3 jax>=0.4.0          # Para GPT-J (mesh-transformer-jax)
4 flax>=0.7.0
5
6 # Data
7 numpy>=1.24.0
8 pandas>=2.0.0
9
10 # Visualization
11 matplotlib>=3.7.0
12 seaborn>=0.12.0
13
14 # Analysis
15 scipy>=1.10.0
16 scikit-learn>=1.2.0
17
18 # LLM evaluation
19 openai>=1.0.0
20 anthropic>=0.30.0
21
22 # Utilities
23 tqdm>=4.65.0
24 pyyaml>=6.0
25 wandb>=0.15.0      # Logging (opcional)
```

0.12. Notas sobre la Reconstrucción

El código descrito en este informe es una **reconstrucción detallada** basada en la descripción del paper. El repositorio original en anonymous.4open.science/r/number-and-letter-neurips2 no fue accesible directamente debido a restricciones de red, pero el paper proporciona información suficientemente detallada para reconstruir la implementación completa:

1. **Arquitectura del modelo** (Appendix C): descripción completa del GPT-J con single-head.
2. **Generación de datos** (Appendix B): definiciones formales y detalles de implementación.
3. **Métricas** (Appendix D): definiciones formales de positional y symbolic scores.
4. **Construcciones teóricas** (Appendix F): pseudo-código y demostraciones completas.
5. **Evaluación en LLMs** (Appendix H): prompts utilizados.
6. **Configuración experimental** (Appendix I): recursos de cómputo.